

Java Backend Interview Handbook

Java Backend Interview Handbook — Volume 1

Target: SDE-1 · Java + Spring Boot · Startup Backend Interviews **Covers:** JVM, Memory & GC, Wrapper Classes, Static, Access Modifiers, Initialization Blocks, Method Overriding, final, equals/hashCode, HashMap Internals

PART 1 — JVM FUNDAMENTALS

1. JDK vs JRE vs JVM ☆☆☆☆

Definition (2 lines):

- **JVM** — Runs bytecode. The engine.
- **JRE** — JVM + core libraries. Enough to *run* Java programs.
- **JDK** — JRE + compiler (`javac`) + dev tools. Needed to *write* Java programs.

JDK ⊃ JRE ⊃ JVM
(develop) (run) (execute bytecode)

Why Java is platform independent: `javac` compiles `.java` → `.class` (bytecode). Bytecode is the same on every OS. Each OS has its own JVM that translates bytecode → machine code.

“Write Once, Run Anywhere” — the **bytecode** is portable, the **JVM** is platform-specific.

Execution Flow:

Hello.java → `javac` → Hello.class (bytecode) → JVM → Machine Code

30-Second Answer: > “JDK is for development — it contains the compiler and tools. JRE is for running programs — it contains the JVM and libraries. JVM actually executes the bytecode. Java is platform independent because bytecode is universal, while the JVM is built per platform.”

Rapid Fire:

Q	A
Is JVM platform independent?	✗ No — JVM is platform <i>dependent</i> ; bytecode is independent
Can we run Java with only JRE?	✓ Yes (run, not compile)
What does <code>javac</code> produce?	Bytecode (.class file)

MCQ: Which component converts bytecode to machine code? A. `javac` B. JRE C. **JVM (JIT/Interpreter)** ✓ D. JDK

2. Class Loader & Parent Delegation ☆☆☆

Definition: Class Loader loads `.class` files into JVM memory when the class is first used.

Three loaders:

1. **Bootstrap** — loads core Java classes (`java.lang.*` , rt/base modules)
2. **Platform (Extension)** — loads platform/extension classes
3. **Application** — loads your classes from classpath

Parent Delegation Model:

Application → asks Platform → asks Bootstrap
Bootstrap tries first ↓ if not found, child tries

A child loader always delegates upward first. Only if the parent can't find the class does the child load it.

Why? Security. You can't create your own `java.lang.String` and trick the JVM — Bootstrap always loads the real one first.

30-Second Answer: > “Class loading follows parent delegation — every loader first asks its parent. Bootstrap loads core classes, Platform loads extensions, Application loads our classpath classes. This prevents malicious replacement of core classes like `String`.”

MCQ: You create your own `java.lang.String` class. What happens? A. Your class loads B. **JDK's String loads (parent delegation)** ✓ C. Runtime error D. Both load

PART 2 — JVM MEMORY & GARBAGE COLLECTION

3. Stack vs Heap ★★★★★

	Stack	Heap
Stores	Local variables, method frames, references	Objects, instance variables
Per thread?	✓ Each thread has its own	✗ Shared by all threads
Lifetime	Until method ends	Until GC collects
Error	StackOverflowError	OutOfMemoryError
Speed	Faster	Slower

```
void fun() {  
    int x = 10;           // Stack  
    User u = new User();  // 'u' reference → Stack, User object → Heap  
}
```

Method Area / Metaspace — stores class metadata, static variables, method bytecode. In Java 8+, PermGen was replaced by **Metaspace** (uses native memory, grows dynamically).

30-Second Answer: > “Stack stores method frames and local variables — one per thread, freed when the method returns. Heap stores all objects and is shared across threads, cleaned by Garbage Collection. Class metadata and static variables live in Metaspace.”

Rapid Fire:

Q	A
Where do objects live?	Heap
Where does a reference variable live?	Stack (if local)
Infinite recursion causes?	StackOverflowError
Heap full causes?	OutOfMemoryError
Where do static variables live?	Metaspace / Method Area (referenced objects on Heap)

4. Heap Generations & Garbage Collection ★★★★★

Heap layout:

```
Heap  
├── Young Generation  
│   ├── Eden        ← new objects born here  
│   ├── Survivor S0  
│   └── Survivor S1  ← survivors bounce between S0/S1  
└── Old Generation  ← long-lived objects promoted here
```

Flow:

1. New object → **Eden**
2. Eden fills → **Minor GC** → survivors move to Survivor space
3. Object survives several Minor GCs → promoted to **Old Gen**
4. Old Gen fills → **Major/Full GC** (slower, causes longer pauses)

	Minor GC	Major GC
Cleans	Young Gen	Old Gen (Full GC = whole heap)
Frequency	Frequent	Rare
Pause	Short	Long

Why generations? Most objects die young. Cleaning only Young Gen frequently is cheap; scanning the whole heap every time would be slow.

When is an object eligible for GC? When it has **no reachable reference** — e.g. `u = null;` or reference reassigned, or object created inside a finished method.

30-Second Answer: > “The heap is split into Young and Old generations because most objects die young. New objects go to Eden; Minor GC clears Eden frequently and cheaply. Objects that survive multiple Minor GCs get promoted to Old Gen, which is cleaned by the slower Major GC. An object becomes GC-eligible when no live reference points to it.”

MCQ: Where is a newly created object allocated? A. Old Gen B. **Eden** ✓ C. Survivor D. Stack

MCQ: Frequent Full GCs in production usually indicate? A. Fast app B. **Memory pressure / leak / undersized heap** ✓
C. Good hashCode D. Nothing

5. Weak References & Memory Leaks ☆☆☆

Reference types:

Type	GC behavior
Strong (<code>User u = new User()</code>)	Never collected while reachable
Weak (<code>WeakReference<User></code>)	Collected at next GC if only weakly reachable
Soft	Collected only when memory is low (good for caches)

```
WeakReference<User> ref = new WeakReference<>(new User());
System.gc();
ref.get(); // likely null – object collected
```

Use case: `WeakHashMap` — entries auto-removed when the key has no strong reference. Prevents cache-based memory leaks.

Common memory leak causes (interview favorite): static collections that keep growing, unclosed resources, listeners never deregistered, long-lived caches holding strong references.

30-Second Answer: > “A weak reference doesn’t prevent GC — if an object is only weakly reachable, it’s collected on the next GC cycle. It’s used in caches like `WeakHashMap` to avoid memory leaks. Java also has soft references, which are cleared only under memory pressure, making them better for caching.”

PART 3 — CORE JAVA FUNDAMENTALS

6. Wrapper Classes ☆☆☆☆

Definition: Object versions of primitives. Needed because Collections, Generics, and Streams work only with objects.

Primitive	Wrapper
int	Integer
double	Double
char	Character
boolean	Boolean

```
ArrayList<Integer> list = new ArrayList<>(); // ✓  
// ArrayList<int> ✗ not allowed  
  
Integer num = 10; // Autoboxing (primitive → object)  
int x = num;      // Unboxing (object → primitive)
```

30-Second Answer: > “Wrapper classes are object representations of primitives, required by Collections and Generics. Java converts between them automatically via autoboxing and unboxing.”

7. valueOf() vs parseXxx() vs xxxValue() ☆☆☆☆

Method	Type	Input → Output
Integer.valueOf("100")	static	String → Integer object
Integer.parseInt("100")	static	String → int primitive
num.intValue()	instance	Integer object → int primitive

```
Integer obj = Integer.valueOf("100"); // object  
int prim   = Integer.parseInt("100"); // primitive  
int back   = obj.intValue();          // object → primitive
```

Why valueOf() over new Integer() ? new Integer() always creates a new object (deprecated). valueOf() reuses **cached objects for -128 to 127** — memory efficient.

Trick question ☆:

```
Integer a = 127, b = 127;  
System.out.println(a == b); // true (cache)  
  
Integer c = 128, d = 128;  
System.out.println(c == d); // false (new objects) — use .equals()!
```

Rapid Fire:

Q	A
valueOf() static?	✓ Yes
parseInt() returns?	primitive int
intValue() static?	✗ No, instance method
Integer cache range?	-128 to 127

MCQ: Integer a = 128; Integer b = 128; a == b prints? A. true B. **false** ✓ C. Compile error D. Exception

8. Static Variables and Methods ☆☆☆☆

Static variable — belongs to the class. **One copy** shared by all objects. Stored in Metaspace.

Static method — belongs to the class. Call via `ClassName.method()`, no object needed.

```
class Example {
    int x;           // per object (Heap)
    static int y;     // one copy (class level)

    static void fun() {
        y = 4;        // ✓ static → static
        // x = 4;      // ✗ Compile error: no object exists yet
    }
    void fun2() {
        x = 4; y = 4; // ✓ instance method accesses BOTH
    }
}
```

Access rules (memorize):

From ↓ can access →	Instance members	Static members
Instance method	✓	✓
Static method	✗ (needs object ref)	✓

Why can an instance method access static? An object always knows its class — so it can see class-level members.

Why can't a static method access instance variables? When the static method runs, there may be **no object** — Java wouldn't know *which* object's variable to use.

Why is main() static? So the JVM can call it directly **without creating an object**.

Static Inner Class:

```
class Example {
    static class Test {
        static String country = "India";
    }
}
// Access: Example.Test.country
```

Real-life analogy: Bank account — `balance` is instance (each customer differs), `interestRate` is static (same for all).

30-Second Answer: > “Static members belong to the class, not objects — one shared copy. Static methods can be called without an object, so they can directly access only static members. Instance methods run on an object, so they can access both instance and static members. `main()` is static so the JVM can invoke it without creating an object.”

MCQ: How many copies of a static variable exist for 100 objects? A. 100 B. **1** ✓ C. 0 D. 2

MCQ: A static method can directly access? A. Instance variables B. **Static variables** ✓ C. this D. super

9. Access Modifiers ★★★★★

Order: `private` < `default` < `protected` < `public`

Modifier	Same class	Same package	Subclass (other pkg)	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

For classes:

- **Outer (top-level) class** → only `public` or `default`. `private`/`protected` outer class ✗.
- **Inner class** → all four allowed (it's a class *member*, like a field).

Public class rules (asked a lot):

1. Only **one public class** per `.java` file.

2. **File name must match** the public class name.
3. Only a public class is directly accessible from **another package**.

Trick: Can a file compile with *no* public class? ✓ Yes — all default classes, any file name.

Private static access:

```
class Example {
    private static int z;
    public static void fun2() { z = 4; }    // ✓ inside class
}
// Example.z = 4;           ✗ private
// Example.fun2();         ✓ modify z via public method
```

30-Second Answer: > “Java has four access levels: private (class only), default (package), protected (package + subclasses), public (everywhere). A top-level class can only be public or default; an inner class can use all four because it’s treated as a member. Each file can have at most one public class, and the file name must match it.”

MCQ: Which is invalid for an outer class? A. public B. default C. **private** ✓ D. Both public & default valid

10. Initialization Blocks ☆☆☆

Two types: Instance Initialization Block (IIB) and Static Initialization Block (SIB).

Instance Initialization Block

Runs **every time an object is created, before the constructor**.

```
class Test {
    int x;
    { System.out.println("IIB"); x = 5; }    // IIB
    Test() { System.out.println("Constructor"); }
}
new Test();    // Output: IIB → Constructor
new Test();    // Output: IIB → Constructor (runs per object)
```

- Multiple IIBs → run in written order, all merged at the start of the constructor.
- ✗ return not allowed. ✓ this and super allowed.
- No constructor? Default constructor is added; IIB still runs.

Static Initialization Block

Runs **once, when the class is loaded** — before first object, static method call, or static variable access.

```
class Test {
    static int k;
    static { k = 10; }    // runs once at class load
}
```

- ✗ return not allowed. ✗ this / super not allowed (no object exists).
- Purpose: initialize static variables / one-time setup.

Execution order (memorize) ☆:

```
Class Load → Static Block (once)
Object Create → Instance Block → Constructor (every object)
```

30-Second Answer: > “A static block runs once when the class loads — used to initialize static variables; this and super aren’t allowed since no object exists. An instance block runs before the constructor every time an object is created — used for common initialization across constructors.”

MCQ: When does a static block execute? A. Every object B. **Once at class load** ✓ C. Every method call D. After constructor

MCQ: Order for new Test() (class already loaded)? A. Constructor → IIB B. **IIB → Constructor** ✓ C. SIB → Constructor → IIB D. Random

PART 4 — OOP: OVERRIDING & FINAL

11. Method Overriding & Runtime Polymorphism ★★★★★

Definition: Child class provides its own implementation of a parent method with the **same signature**.

```
class A { void show() { System.out.println("A"); } }
class B extends A {
    @Override void show() { System.out.println("B"); }
}

A obj = new B();
obj.show();           // Output: B
```

Why “B”? Runtime polymorphism (Dynamic Method Dispatch) — JVM picks the method based on the **actual object type (B)**, not the reference type (A). Decided at **runtime**.

Rules:

- ✓ Same name, same parameters, same/covariant return type
- ✗ Cannot **reduce** visibility (public → private ✗; can widen: protected → public ✓)
- ✗ Cannot override: static, final, private, constructors

Why can’t static methods be overridden? Static methods belong to the **class**, resolved at **compile time** by reference type. Redefining one in a child is **method hiding**, not overriding.

```
class A { static void f() { System.out.println("A"); } }
class B extends A { static void f() { System.out.println("B"); } }
A obj = new B();
obj.f();           // Output: A ← reference type decides (hiding!)
```

Why @Override ? Compiler verifies you actually override — typo in signature = compile error instead of a silent bug.

Overriding vs Overloading:

	Overriding	Overloading
Signature	Same	Different parameters
Resolved	Runtime	Compile time
Requires inheritance	✓	✗
Polymorphism type	Runtime	Compile time

30-Second Answer: > “Overriding lets a child class redefine a parent method with the same signature, enabling runtime polymorphism — the JVM calls the method based on the actual object’s type, not the reference type. Static methods can’t be overridden, only hidden, because they’re resolved at compile time by reference type.”

MCQ: A obj = new B(); obj.show(); (show overridden in B) prints? A. A B. **B** ✓ C. Compile error D. Both

MCQ: Static method redefined in a child class is called? A. Overriding B. **Method Hiding** ✓ C. Overloading D. Shadowing

12. final Keyword ★★★★★

One word, three protections:

Applied to	Effect
Variable	Value/reference assigned only once
Method	Cannot be overridden (can be inherited & overloaded)
Class	Cannot be extended (e.g., String)

final Instance Variable — 3 ways to initialize

```

final int a = 10;           // 1. Declaration
final int b; { b = 10; }    // 2. Instance init block
final int c; Example() { c = 10; } // 3. Constructor (most common)

```

Blank final = declared without value → compiler forces exactly one assignment (constructor/block), else compile error.

final static Variable — class constant

```

static final int MAX;           // blank static final
static { MAX = 100; }           // ✓ static block
// Example() { MAX = 100; }     // ✗ constructor NOT allowed

```

Why not the constructor? Static variables are initialized at **class load**, before any object/constructor exists.

	final instance	final static
Copies	One per object	One per class
Init in	Declaration / IIB / Constructor	Declaration / Static block

final Local Variable

```

void fun() {
    final int k = 10;
    // k = 20; ✗
}

```

Must be initialized before use; can't be reassigned. Important for **lambdas** — captured locals must be final or *effectively final*.

final Method

```

class A { final void show() { System.out.println("A"); } }
class B extends A {
    // void show() {}           ✗ Compile error – cannot override
    void show(int x) {}        // ✓ Overloading IS allowed
}
new B().show();               // ✓ Inherited normally → prints "A"

```

final Class

```

final class Dummy { }
// class Child extends Dummy {} ✗

```

Real example: `String` is final — guarantees security & consistent behavior.

★ Trap: Does final make an object immutable? — NO

`final` locks the **reference (the arrow)**, not the object.

```

final List<String> list = new ArrayList<>();
list.add("Java");           // ✓ object state changes
// list = new ArrayList<>(); // ✗ reference can't change

final StringBuilder sb = new StringBuilder("Java");
sb.append(" Spring");
System.out.println(sb);     // "Java Spring" – even though sb is final!

```

Memory trick: Dog on a final leash — the dog can still bark and move (object mutates); you just can't tie the leash to another dog (reference fixed).

True immutability = class design:

```

final class Employee {           // no subclass
    private final String name;    // final field, no setter
    Employee(String name) { this.name = name; }
    public String getName() { return name; }
}

```

```
}
```

That's why `String` is immutable — `s.concat("x")` returns a **new** `String`, never modifies the original.

30-Second Answer: > “final on a variable means one-time assignment, on a method means it can't be overridden, on a class means it can't be extended. Importantly, a final reference doesn't make the object immutable — a final `ArrayList` can still be modified; only the reference can't point elsewhere. True immutability comes from class design, like `String`: final class, final private fields, no setters.”

Rapid Fire:

Q	A
final method inherited?	✔ Yes
final method overloaded?	✔ Yes
final method overridden?	✗ No
static final init in constructor?	✗ No
final List — can add()?	✔ Yes
Is String final?	✔ Yes

MCQ: `final StringBuilder sb = new StringBuilder("Hi"); sb.append("!");` → A. Compile error B. Runtime error C. **Prints Hi!** ✔ D. Prints Hi

MCQ: Blank static final variable can be initialized in? A. Constructor B. **Static block** ✔ C. Instance block D. Any method

PART 5 — equals()/hashCode() & HashMap INTERNALS

13. equals() / hashCode() Contract ★★★★★

Default behavior: `Object.equals()` compares **memory addresses** → two `new User(1)` objects are NOT equal until you override `equals()`.

The Contract (memorize all 3):

1. `a.equals(b) == true` → `a.hashCode() == b.hashCode()` **MUST** be true
2. Same hashCode → may or may not be equal (**collision** — resolved by `equals()`)
3. Different hashCodes → definitely **not equal**

Why the contract exists — HashMap lookup:

```
put(key): hashCode() → bucket → store
get(key): hashCode() → bucket → equals() within bucket → found
```

If equal objects have different hashCodes, `get()` searches the **wrong bucket** and `equals()` is never even reached → returns `null`.

The classic trap ★★★★★:

```
class User {
    int id;
    User(int id) { this.id = id; }
    @Override public boolean equals(Object o) {
        return ((User) o).id == this.id;
    }
    // hashCode() NOT overridden!
}

HashMap<User, String> map = new HashMap<>();
map.put(new User(1), "Developer");
System.out.println(map.get(new User(1))); // null !
```

Why null? Both objects get different **default** hashCodes → put stored in bucket X, get searched bucket Y. Fix: override `hashCode()` too (`Objects.hash(id)`).

30-Second Answer: > “`hashCode()` decides the bucket; `equals()` finds the exact key inside the bucket. So if two objects are equal by `equals()`, they must return the same hashCode — otherwise HashMap looks in the wrong bucket and lookups fail. That’s why we always override both together.”

MCQ: `equals()` overridden, `hashCode()` not. `map.get(equalKey)` returns? A. Value B. **null** ✓ C. Exception D. Compile error

MCQ: Two objects have the same hashCode. They are? A. Definitely equal B. **May or may not be equal** ✓ C. Never equal D. Same object

14. HashMap Internal Working ★★★★★

Internals in one flow:

```
put(key, value)
→ key.hashCode()
→ index = hash & (capacity - 1)
→ bucket[index]
→ empty? store : collision → equals() check
→ same key? replace value
→ new key? append to LinkedList (or Tree)
```

Key numbers (memorize):

Item	Value
Default capacity	16

Load factor	0.75
Resize trigger	$16 \times 0.75 = 12$ entries
List → Red-Black Tree (Java 8+)	bucket size > 8 (and capacity ≥ 64)
Complexity	Avg $O(1)$, worst $O(n) \rightarrow O(\log n)$ with tree

Resizing (rehashing): capacity doubles, all entries recomputed to new buckets — **expensive**.

Why is String the best key? Immutable + correct hashCode/equals + hash caching.

Production Scenarios (Backend-Lead style)

Q1. HashMap with millions of entries became slow. What do you check? Structured answer:

1. **Collision rate** — poor hashCode() → all keys in few buckets → $O(n)$ lookups
2. **Resizing** — default capacity 16 → millions of inserts = many expensive rehashes
3. **GC pressure** — millions of objects → frequent GC pauses
4. **Concurrency** — multiple threads? → use `ConcurrentHashMap`

Q2. Every key returns the same hashCode()? All entries land in one bucket. Map still works correctly (equals() distinguishes keys) but degrades $O(1) \rightarrow O(n)$ (Java 8+: $O(\log n)$ after treeification).

Q3. Mutable key modified after put()?

```
map.put(user, "Dev"); // stored using hash(id=1)
user.id = 2;          // hash changes!
map.get(user);        // null - searches wrong bucket
```

Entry is “lost”. That’s why keys should be **immutable** (String, Integer).

Q4. You’ll store 10 million records — how to initialize?

```
new HashMap<>(13_500_000); // ≈ expected / 0.75
```

Pre-sizing avoids repeated resize + rehash of millions of entries.

30-Second Answer: > “HashMap computes the key’s hashCode to pick a bucket, then uses equals() to find the exact key within it. Default capacity is 16 with load factor 0.75, so it resizes at 12 entries. Collisions form a linked list, which becomes a red-black tree beyond 8 nodes in Java 8+. For performance issues I’d check collision quality, resizing frequency, GC pressure, and thread safety.”

Rapid Fire:

Q	A
Default capacity / load factor	16 / 0.75
Collision handled by	equals() in bucket (chaining)
Java 8 improvement	Treeify long chains → $O(\log n)$
Thread-safe alternative	<code>ConcurrentHashMap</code>
Best key type	Immutable (String)
Can HashMap have a null key?	✔ One null key allowed

MCQ: Default HashMap resizes after how many entries? A. 16 B. **12** ✔ C. 8 D. 32

MCQ: All keys return hashCode() = 1. HashMap will? A. Throw exception B. Lose data C. **Work correctly but slower** ✔ D. Store only 1 entry

FINAL 2-MINUTE REVISION SHEET ⚡

Topic	One-liner
JDK/JRE/JVM	Develop / Run / Execute bytecode
Platform independence	Bytecode universal, JVM per-OS
Class loading	Parent delegation — Bootstrap first (security)
Stack vs Heap	Method frames per-thread vs shared objects
GC generations	Most objects die young → cheap Minor GC in Eden
Weak reference	Doesn't block GC — used in WeakHashMap caches
valueOf vs parseInt	Object vs primitive; Integer cache -128..127
Static	One copy, class-level; static method can't touch instance state
main() static?	JVM calls it without creating an object
Access modifiers	private < default < protected < public; outer class: public/default only
Public class rules	1 per file, filename matches, only public crosses packages
Static block	Once at class load; no this/super/return
Instance block	Before constructor, every object
Overriding	Runtime polymorphism — actual object type wins
Static override?	No — hiding, resolved by reference type
final variable	One assignment; reference locked, object still mutable
final method / class	No override / no extend; String is final
Blank static final	Init in declaration or static block only
equals/hashCode	Equal objects ⇒ same hashCode, else HashMap breaks
HashMap	hashCode → bucket, equals → key; 16/0.75/12; treeify >8
Mutable key	Hash changes → entry lost → use immutable keys

Interview Delivery Tips ☆

1. **Structure answers:** “First I’d check X, then Y, then Z” — sounds like production experience.
2. **Always say WHY**, not just what. (“main is static *because* the JVM needs to call it without an object.”)
3. When shown code, **predict output first**, then explain the mechanism.
4. If unsure, reason out loud from fundamentals — leads respect the thought process.
5. For your 6 months’ experience: connect answers to real work (“we used ConcurrentHashMap because...”).

All the best, Sagar. Crack it. 🚀